

App Team — Mosquito Risk Monitoring MVP

Full Data Integration Brief | unoff_app

1. Architecture Reminder

This is a single unified system. The three data layers feed each other directly:

Layer	Team	Format	Role
Base signals	DataTeam	GeoJSON (Point)	Sentinel-2 spectral indices per pixel
Processed indicators	FeatureTeam	GeoJSON (Polygon)	CV-derived water / veg / shadow scores
Decision layer	RiskTeam	GeoJSON + JSON	Final risk categories on 20x20 grid

Default view: block-level risk grid (RiskTeam decision layer)

On cell click: inspection panel opens with FeatureTeam + DataTeam overlays

Repo: https://github.com/Darkegale12/unoff_app (Vite + React, Tailwind, src/ structure)

2. Layer 1 — Default Map: Risk Decision Grid (RiskTeam)

Source files

- week01_pre.geojson ... week10_post.geojson — 400 polygon features each (20x20 grid)
- week_01.json ... week_10.json — full state with risk_grid, uncertainty_grid, decision_grid (20x20 float arrays)
- day_001.json ... day_070.json — daily granularity

GeoJSON feature properties to consume

```
{
  "cell_id": "R00C00",
  "row": 0, "col": 0, "week": 1,
  "avg_risk": 0.48,
  "risk_category": 1,
  "risk_category_label": "Medium",
  "phase": "pre"
}
```

Color mapping

Low (0)

Blue — #4393c3

Medium (1)

Orange — #f4a35a

High (2)

Red — #d6312b

Time control

- Weekly slider (Wk1–Wk10) switching between week_XX.json files
- Optional daily slider from day_XXX.json files (day 1–70)
- Reference implementation: map3_daily_weekly_slider.html in RiskTeam zip (Leaflet.js, onTimeChange function, showCellInfo handler)

3. Layer 2 — On Click: Advanced Inspection Panel

When a user clicks a grid cell, a dedicated side panel opens. The panel always shows:

- Pixel-level feature overlays (FeatureTeam)
- Supporting spectral signals (DataTeam)
- Risk context for that cell (RiskTeam)
- Drone video feed — placeholder, future integration

4. Data Schemas

RiskTeam — week_XX.json

```
{
  "week": 1, "week_label": "Wk1",
  "date_start": "2024-06-01", "date_end": "2024-06-07",
  "phase": "pre",
  "climate": {
    "temperature_c": 28.0, "rainfall_mm": 5.0,
    "humidity_pct": 55.0, "activity_multiplier": 0.5469
  },
  "risk_summary": { "mean_risk": 0.1789, "n_high_cells": 0, ... },
  "impact": { "risk_before": 0.1946, "risk_after": 0.1937, "reduction_pct": 0.5 },
  "risk_grid": [[0.48, 0.28, ...], ...], // 20x20 floats 0-1
  "uncertainty_grid": [[...], ...], // 20x20 variance
  "decision_grid": [[...], ...] // 20x20 category int
}
```

RiskTeam — day_XXX.json

```
{
  "day": 1, "date": "2024-06-01", "phase": "pre",
  "climate": { "temperature_c": 28.0, "rainfall_mm": 5.0, ... },
  "risk_summary": {
    "mean_risk": 0.1946, "n_high_cells": 16,
    "n_medium_cells": 158, "convergence_score": 0.0627
  },
  "sensor_positions": [
    { "sensor_id": 1, "row": 6, "col": 5,
      "lat": 18.463158, "lon": 73.852632 }
  ],
  "risk_grid": [[...], ...],
```

```
"intervention": { "triggered": false, "reason": "...", "zones": [] }
}
```

DataTeam — {YYYY-MM-DD}.geojson

Path pattern: Lotus_Organized/{YEAR}/Month_{MM}/{BAND}/{YYYY-MM-DD}.geojson

Bands: NDVI, NDWI, NDBI, MNDWI, NDMI, LST | ~8,985 Point features per file

```
{
  "type": "FeatureCollection",
  "features": [{
    "type": "Feature",
    "geometry": { "type": "Point", "coordinates": [lon, lat] },
    "properties": {
      "timestamp": "2022-01-14",
      "source": "sentinel-2",
      "features": { "ndvi": 0.282 },
      "quality": { "valid": true }
    }
  }
]}
}
```

FeatureTeam — CV Pipeline GeoJSON

Generated by: main.py --video <drone_video.mp4> Output: outputs/geojson/<frame_id>.geojson

```
{
  "type": "FeatureCollection",
  "features": [{
    "type": "Feature",
    "geometry": { "type": "Polygon", "coordinates": [[[...]]] },
    "properties": {
      "frame_id": "...", "cell_id": "...",
      "water": 0.72, "vegetation": 0.45,
      "shadow": 0.21, "water_proximity": 0.80,
      "risk_proxy": 0.65
    }
  }
]}
}
```

5. Inspection Panel — Component Spec

Suggested React component signature

```
<InspectionPanel
  selectedCell={{ row, col, cell_id, lat, lon }}
  riskData={{
    avg_risk, risk_category_label,
    uncertainty, phase
  }}
  featureData={{
    water, vegetation, shadow,
    water_proximity, risk_proxy
  }}
  spectralData={{ ndvi, ndwi, ndbi, lst }}
  videoUrl={null} // drone feed – coming soon
/>
```

Panel layout

- Fixed/persistent side area — not a popup — always visible once a cell is selected
 - Updates reactively when a different cell is clicked or the time slider moves
 - Shows three data layers merged by location: Feature scores + Spectral bands + Risk values
 - Visual indicator (bar or color chip) for each numeric score
 - Drone video slot: render a <video> tag or placeholder div with label 'Drone feed (coming soon)' — accept a videoUrl prop
 - Panel re-fetches data for the current selected cell when week/day changes
-

6. Integration Steps

Step 1 — Serve data files

```
Host or bundle as static assets (or lightweight API):
RiskTeam/week_XX.json (10 files)
RiskTeam/weekXX_pre/post.geojson (10 files) — for map rendering
RiskTeam/day_XXX.json (70 files) — for daily slider
DataTeam/Lotus_Organized/{year}/Month_{mm}/{band}/{date}.geojson — load on demand
FeatureTeam/outputs/geojson/*.geojson — load on demand per cell click
```

Step 2 — Render the Risk Map

```
Use Leaflet.js (reference: map3_daily_weekly_slider.html) or react-leaflet.
Render weekly GeoJSON with category coloring.
Bind onClick per feature to populate inspection panel state.
```

Step 3 — Build the Inspection Panel

```
Implement the InspectionPanel component per the spec in Section 5.
Three data sub-sections: Feature Scores, Spectral Signals, Risk Context.
Include placeholder drone video slot.
```

Step 4 — Data loading on cell click

```
Read row, col, lat, lon from the GeoJSON feature properties.
From week_XX.json / day_XXX.json: extract risk_grid[row][col], uncertainty_grid[row][col].
Load nearest DataTeam GeoJSON files for the current time period (match by lat/lon).
If FeatureTeam GeoJSON is available, load and filter by cell bounding box.
```

Step 5 — Time slider

```
Reference the onTimeChange function in map3_daily_weekly_slider.html.
On slider move: swap the active JSON, re-render the risk grid,
and update the inspection panel if a cell is already selected.
```

7. Coordinate Reference

The RiskTeam 20x20 grid covers approximately Pune, India:

- Lat range: ~18.40 – 18.60 N
- Lon range: ~73.80 – 73.99 E
- Each cell $\approx 0.01^\circ \times 0.01^\circ$

Cell (row, col) to lat/lon centre approximation:

```
lat = 18.4 + (row * 0.01) + 0.005
lon = 73.8 + (col * 0.01) + 0.005
```

Use this formula to spatial-join DataTeam and FeatureTeam GeoJSON features to the clicked cell.

8. Work Summary

Component	Status	Notes
Risk map with time slider	Reference exists	Port Leaflet logic from map3_daily_weekly_slider.html to React
Cell click handler	New	Set selected cell state, trigger data loads
Inspection Panel (side area)	New	Always visible, updates on cell click
Feature layer in panel	New	Reads FeatureTeam GeoJSON outputs
Spectral signals in panel	New	Reads DataTeam GeoJSON on demand
Risk context in panel	New	Reads from active week_XX / day_XXX JSON
Drone video slot	Placeholder	videoUrl prop, 'coming soon' label

This document was auto-generated from DataTeam.zip, FeatureTeam.zip, RiskTeam.zip, and the unoff_app GitHub repo.